

Advancing Near-Future Performance Prediction in Cloud Systems Using Deep Learning

1nd Sowmya Chintakindi
Sr. Performance Engineer
Oklahoma, USA
sowmyar909@gmail.com

2nd Bharath Kumar Maganti
Sr. Performance Engineer
Austin, USA
kumar.mbk2110@gmail.com

Abstract—As technology is evolving rapidly, it is required to 99.99% uptime with optimal performance to meet current user demands. To meet this demand, Deep learning framework plays a key role in predicting unplanned performance bottlenecks that can disrupt operations and lead to costly recovery. To reduce recovery time and better prepare for potential crashes, this work focuses on predicting future system performance metrics with the goal of maintaining 99.99% uptime. A time-series prediction model was built using a PyTorch-based LSTM and trained on performance testing data. In production, the model can use real-time metrics to forecast potential failures in advance. During training, the model was run only for 30 epochs, with loss steadily decreasing from approximately 18766 in the first epoch to about 18647 by the final epoch, indicating that the model was learning and improving its predictions over time.

Index Terms—Deep Learning Framework, AWS Sagemaker, PyTorch, Predictions, Performance bottlenecks

I. INTRODUCTION

It is required for organizations to maintain optimal system performance, which requires the ability to anticipate issues before they impact users. Predicting near-future performance metrics can help teams proactively manage capacity, detect anomalies, and prevent outages. In this study, building such predictive models using deep learning framework is constrained by limited access to real-time production data due to privacy, cost, and operational restrictions.

To address this challenge, we demonstrate how ML models can be leveraged to predict near-future system behavior using data generated through controlled performance testing. Instead of relying on live production data, a REST API was developed using the Python language and deployed using a lambda/serverless architecture, making it accessible over the internet. Synthetic performance data was then generated by applying load through JMeter, simulating realistic usage patterns at scale. Latency metrics were captured in AWS CloudWatch and fed to AWS SageMaker to train a Long Short-Term Memory (LSTM) model implemented in PyTorch and deployed on SageMaker to predict near future latency metrics.

We demonstrated a fully implemented end-to-end cloud AI pipeline, including workload generation using JMeter, metric capture using CloudWatch, model training, deployment, and inference using AWS SageMaker. The results highlight a practical and repeatable approach for applying AI-driven per-

formance forecasting in cloud systems, supporting proactive system management and improving service reliability.

This paper starts with the literature review of research papers in similar tracks in section 2, followed by the research methodology in section 3. Section 4 provides an understanding of the architecture and environment setup used in this study. Section 5 provides the results of the experiments conducted. This is followed by the analysis of the results in section 6. Conclusions are presented in section 7.

II. RELATED WORK AND LITERATURE REVIEW

The authors of the paper [16] propose a framework for predictive maintenance in cloud environments with the help of AWS CloudWatch combined with machine learning algorithms for advanced failure prediction and anomaly detection. Key components include data ingestion from CloudWatch, feature engineering, model training using algorithms suited for time-series and anomaly tasks. The paper lacks providing the results on using this framework and predicting near future performance metrics.

Authors of the paper [13] have primarily focused on flexible, model-agnostic forecasting frameworks for system metrics. These frameworks are treated as a service capability, not a model-centric study. These approaches do not explore deep learning-based time-series modeling. In contrast, our study specifically targets near-future latency prediction, leveraging a PyTorch-based LSTM model to capture temporal dependencies.

The paper [8] proposes a hardware-centric anomaly detection framework that often spans hardware, software, networks, and applications without giving operators direct workload visibility. It uses an unsupervised learning pipeline developed by analyzing over 30 popular ML models across diverse hardware platforms to identify anomalous behavior indicative of performance bottlenecks or configuration issues without needing knowledge of the actual workload, however, it does not predict near future performance bottlenecks or anomalies.

In the paper [10], performance bottleneck detection was conducted using Google Cloud based AI models under varying workloads. The current work extends that research by leveraging AWS-native services to predict near-future performance metrics.

III. RESEARCH METHODOLOGY

This study follows a quantitative experimental approach conducted using an AWS instance and open source tools. The objective is to identify bottlenecks in realtime when a performance test is done using ML models.

A. Experimental Approach

Developed a RESTful API using Python to perform CRUD operations like Add, get, update, and delete items and made it accessible over the internet by deploying on an AWS Lambda [6] serverless architecture. Induced some performance issues like slow response times and "500 Internal Server" error codes. Executed an hour long performance tests on the API using JMeter [17] with wait time between requests, capturing response times and response codes to evaluate the accuracy of the test results. Latency metrics were captured in AWS Cloudwatch that were fed to SageMaker. This synthetic data was used to train the PyTorch LSTM ML Model in SageMaker and same model was used as inference to predict near future latency metrics.

B. Justification Of Tool Choices

This study follows a structured five-phase approach.

1) *API Development and Deployment*: Developed RESTful API as it is widely used in all types of architecture and is simple to design, test, and integrate with other services. Also, Python [11] is lightweight and allows quick development using frameworks like Flask.

2) *Performance Testing with JMeter*: JMeter[12] was used to simulate load on the Python REST API because of its open-source nature, and it is easy to develop and execute performance tests[9]. It is also used to send requests to Google Analytics via API request.

3) *Serverless Lambda in AWS*: This serverless compute service helps you to run the code without any physical computer or spinning up virtual server on cloud.

4) *Serverless DynamoDB in AWS*: Since serverless Lambda is stateless and can create multiple instances dynamically, at 50 TPS it can have multiple instances, each with its own memory space to store the data. Serverless DynamoDB [3] has been used to share memory across instances.

5) *AWS CloudWatch*: AWS CloudWatch [2] is a monitoring tool provided by AWS for free that gives insights into AWS resources like CPU utilization, time taken for the service/server to respond or latency.

6) *AWS SageMaker*: AWS SageMaker [4] is a fully managed ML service provided by AWS to build, train, and deploy machine learning models with the infrastructure fully managed by AWS.

C. Limitations and Challenges

This study used the AWS free tier Lambda function, DynamoDB, and CloudWatch. Additionally, used SageMaker with instances of size ml.t3.medium to train and predict data based on 1 hour of performance test data, thereby restricting

the ability to conduct extended durations with very high-volume and production data due to privacy, cost, and operational restrictions. As a result, the experiments were carried out under constrained load settings, largely as proof of concept for predicting bottlenecks during test execution.

IV. SYSTEM ARCHITECTURE

This section describes the high level structure of the system used in this study describing how its components are organized, interacted and work together to achieve the study's goal.

A. Python API development

At the application layer, a Python API is developed as AWS Lambda serverless API designed to be triggered by API gateway that integrates with DynamoDB to manage data sent by the function. This API is structured to work seamlessly to return responses in JSON format for both successful operations and errors. This API is accessible to multiple clients, including web applications, mobile apps, and testing pipelines. Lambda function was configured with 128MB memory, 512MB ephemeral storage and 3secs timeout.

This API has a dual-purpose design: a functioning CRUD service for structured data and a performance simulation environment that replicates realistic system characteristics.

- Health Check Endpoint
 - "/" → Returns a simple JSON response indicating that the API is active. This is primarily used to monitor the API's availability.
- CRUD Endpoints

The API simulates a lightweight in-memory database (items =) and provides the standard Create, Read, Update, Delete (CRUD) functionality:

 - POST /items/ → Creates a new item, ensuring unique names.
 - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
 - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
 - PUT /items/"name" → Updates an existing item with new data.
 - DELETE /items/"name" → Deletes an item by name.

This design offers a simple yet extensible data interaction model, suitable for experimentation and integration with downstream services.
- Performance Testing Endpoints

To evaluate system resilience and simulate different conditions, the API provides specialized endpoints:

 - /fast → Returns an immediate response, representing minimal latency.
 - /slow → Introduces a 2-second delay before responding, simulating a slow endpoint.
 - /random → Introduces a random delay (0.1–3.0s) to emulate unpredictable response times.
 - /unstable → Has a 20% probability of failure (returns HTTP 500) and otherwise succeeds, modeling unstable network or server conditions.

- Deployment

The Lambda-deployed API was exposed to the internet through API Gateway [1] posting data to a serverless DynamoDB table, since Lambda functions are stateless and can run across multiple instances. Each cold start produces a fresh execution environment with an empty in-memory dictionary. At a rate of around 50 TPS, Lambda can spawn multiple concurrent instances, each maintaining its own isolated in memory for variables. The steps mentioned below describe the end-to-end deployment workflow for hosting a Python API in serverless architecture.

- Created a Lambda function to deploy the Python API, using the default settings. Test event with default settings was created to deploy changes made to Lambda function.
- Created an API Gateway resource and integrated it with the Lambda function to handle all request methods.
- Enabled Lambda proxy integration to ensure full request and response pass through.
- Attached the policies as mentioned in the figure to allow Lambda publish logs, metrics, and insights to CloudWatch.
- Created a DynamoDB table with Name as primary key and with default settings. Lambda code was later updated to write incoming data to this table.
- Enabled Lambda insights enhanced monitoring under lambda configuration - monitoring and operations tools.

B. Executing Tests with JMeter

This JMeter script [7] was designed to perform CRUD operations: Add, Get, Update, and Delete items using "HTTP Request" element. Test data for the script was injected using CSV Data Set Config element with data containing the fields Name, Description, Price, Quantity, Update item, and End-point. Test was executed on serverless architecture to gather the data with the load slowly ramping up to 50 TPS to sustain at 50 TPS for one hour to train the models. Some of the performance issues like slower transactions and unstable transactions were induced to test the predictability of the models.

All other requests followed the same configuration approach with below mentioned configuration (on port 443), endpoints and empty body.

- Get Item: /Test/items/\$Name
- Get API status with induced performance bottleneck: /Test/ \$Endpoint
- Delete an Item: /Test/items/\$Name

C. Validating Metrics in CloudWatch

API deployed on AWS Lambda publishes its operational metrics to Amazon CloudWatch after the required IAM policies are enabled for both Lambda and API Gateway. In addition to standard metrics Enhanced Lambda Monitoring is

enabled to capture detailed resource-level metrics, including CPU utilization, memory usage, disk, and network activity. These metrics can be viewed in CloudWatch under Lambda Insights, providing comprehensive visibility into the function's performance and resource behavior.

D. Retrieving Performance Metrics to SageMaker

Python monitoring and analysis script was written in SageMaker JupyterLab notebook [5] to retrieve performance data to use for training the models. It queries AWS CloudWatch to collect key performance metrics and organizes the results into a clear, readable format.

E. Preparing Time-Series data

A python code was written that prepares data to use it in training machine learning model such as LSTM. It converts sequence of data points into overlapping input windows and corresponding target values, allowing the model to learn patterns from recent observations and predict the next value in the series. The final output shows how the data has been structured into inputs and labels, confirming that it is ready for training a sequence-based prediction model.

F. PyTorch LSTM Time-Series Training Loop

This python code defines and trains a simple LSTM-based neural network using PyTorch [14] to forecast future values from time-series past data. The training loop repeatedly feeds the retrieved data from CloudWatch into the model, calculates prediction error using mean squared loss, and updates the model parameters through backpropagation and optimization. Over multiple training cycles or epochs, the model gradually improves its ability to predict time-series behavior, with the loss printed each time to track learning progress.

G. Performance Bottlenecks Predictions

Python code written in this section uses trained LSTM model to generate future predictions from time-series data obtained in above section. It repeatedly feeds a sliding window of data into the model to predict the next time step, then appends that prediction back into the input sequence. This process is repeated to forecast multiple future points ahead. Finally, the predicted values are converted back to their original scale and printed, providing an estimate of how the performance metrics may behave in the near future.

V. RESULTS

This section presents the outcomes of the experiments conducted in this study. These results were further analyzed to evaluate the effectiveness of our approach of predicting near future performance metrics using Machine learning model. The results also provides insights into model performance and system behavior laying the foundation for interpreting findings and discussing their implications in the context of the research objectives.

A. API Operational Insights

This section provides results obtained after successful deployment of Python API on serverless architecture. After successful deployment, API is available over internet using API Gateway endpoint URL as shown in the figure below.



Fig. 1. Successful API deployment

B. JMeter Training Data Generation

Requests were generated using Apache JMeter to simulate traffic to the deployed API based on predefined test data containing the fields Name, Description, Price, Quantity, Update item, and Endpoint. The requests were distributed across multiple endpoints, including fast, slow, random (with response times varying between 0.1 and 3 seconds), and unstable (returning intermittent 500 errors). This section presents the results of the load test execution conducted with sustained load of 50 transactions per second (TPS) for one hour, including response time visualizations and results of the intentionally induced performance bottlenecks.

Figure 12 shows response headers and response body for the regular request.



Fig. 2. Snippet Showing Response with Endpoint /

Figure 13,14,15,16 shows response headers and response body for the performance bottleneck induced requests. These requests were sent to perform sanity check on the request to evaluate if the functionality is working as expected.

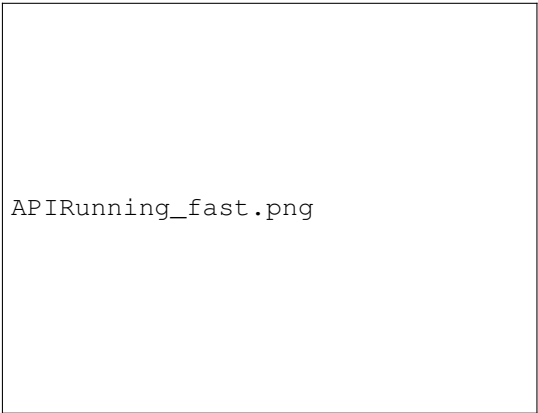


Fig. 3. Snippet Showing Response with Endpoint /fast



Fig. 4. Snippet Showing Response with Endpoint /slow



Fig. 5. Snippet Showing Response with Endpoint /random

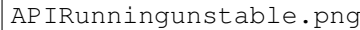
A placeholder image for a screenshot showing a response with the endpoint /unstable.

Fig. 6. Snippet Showing Response with Endpoint /unstable

Load test was executed with the requests shown above with 50TPS for one hour. Figure 17 shows the response times for all the request types from JMeter tool. The scale was adjusted to better visualize lower response time values.

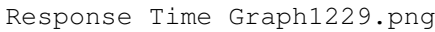
A placeholder image for a screenshot of a response time graph from JMeter.

Fig. 7. Response Time Graph from JMeter

C. Capturing the Metrics in CloudWatch

After enabling lambda insights for each Lambda function and attaching the CloudWatchLambdaInsightsExecutionRole-Policy to the Lambda execution role, Lambda Insights collects CPU utilization, memory usage, network I/O, disk usage, and execution environment metrics. These Metrics are published automatically to Amazon CloudWatch and can be accessed under CloudWatch → Lambda Insights as well as via the GetMetricData API. Data is reported at fine-grained intervals, enabling deeper visibility into function behavior during load testing and fault scenarios.

Figure 18,19 and 20 shows the 99th percentile response times (duration), CPU used and memory used during the test.

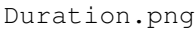
A placeholder image for a screenshot showing response times for all transactions in CloudWatch.

Fig. 8. Response times for all Transactions in CloudWatch

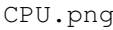
A placeholder image for a screenshot showing CPU used by Lambda Function as per CloudWatch.

Fig. 9. CPU used by Lambda Function as per CloudWatch

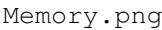
A placeholder image for a screenshot showing memory used by Lambda Function as per CloudWatch.

Fig. 10. Memory used by Lambda Function as per CloudWatch

D. Feeding CloudWatch Metrics to SageMaker

Python script in this section was created to collect AWS Lambda performance metrics such as CPU time, Memory utilization and Duration using Boto3 and Pandas. Desired time stamp was given as input to the script to get the metrics for that time period. The script then queries CloudWatch for three metrics, Duration from Lambda namespace and CPU usage, Memory utilization from Lambda Insights each aggregated over 60 second intervals.

Figure 21 shows some of the samples that were collected from CloudWatch to SageMaker to use it to train the models and predict near future metrics.

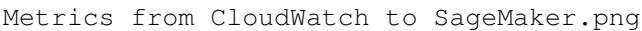
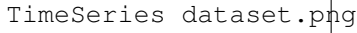
A placeholder image for a screenshot showing some samples of CloudWatch metrics in SageMaker.

Fig. 11. Some samples of CloudWatch Metrics in SageMaker

E. Preparing Time-series Data for LSTM learning

Python code in this section was created to prepare a multivariate time series dataset from a DataFrame containing AWS Lambda metrics: response time, CPU time, and memory utilization. Using a sliding window approach of size 2, it constructs input sequences (X) consisting of consecutive measurements of all three features and target values (y) corresponding to the next timestep for each sequence. The resulting X and y arrays are then converted to NumPy arrays, suitable for training machine learning models like LSTMs or other sequence predictors. Below figure shows the output of this code



TimeSeries dataset.png

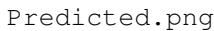
Fig. 12. Datapoints Prepared to Train the Model

F. Model Training

Python code in this section was created to train an LSTM model using PyTorch to predict AWS Lambda performance metrics response time, CPU usage, and memory utilization based on past observations. The ForecastLSTM class is an LSTM with a hidden layer of 32 units, followed by a fully connected layer that outputs predictions for all three metrics at the next timestep. The input X_t is a 3D tensor representing sequences of past measurements over a sliding window, while y_t contains the corresponding target values. The model is trained for 30 epochs using the Adam optimizer and mean squared error loss, updating its weights each epoch to minimize the prediction error across all three metrics. This setup allows the model to capture temporal dependencies and relationships between the metrics for short-term forecasting. Below figure shows the output of Model training script.

G. Predicting Near Future Performance Metrics

Python code in this section predicts near-future AWS Lambda performance metrics using a trained LSTM model in PyTorch. It starts by extracting three key metrics from a DataFrame and scales them to a normalized range using MinMaxScaler, which helps the LSTM model converge more effectively. A sliding window of the last window timesteps is initialized to provide sequential input to the model. The code then enters a loop where at each step the LSTM takes the current value as input, predicts the metrics for the next minute, and the window is updated by removing the oldest row and appending the predicted values, allowing the model to recursively forecast multiple steps ahead. Figure below shows the output of the predicted values using the Model.



Predicted.png

Fig. 13. Predicted Performance Metrics

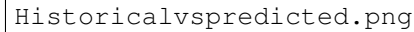
VI. ANALYSIS

JMeter represents client-side response times per transaction, which include network latency, API Gateway overhead and Lambda execution. Whereas, CloudWatch provides server-side response times for the all the transactions together. Although some spikes in JMeter do not align exactly with Lambda execution duration spikes, this behavior is expected due to differences in metric aggregation in reporting latency. Since, JMeter captures instantaneous client-side response times, and CloudWatch captures averaged server-side execution times over fixed intervals with an inherent publishing delay of approximately two seconds. As a result, sudden latency increases may appear temporally shifted in CloudWatch.

SageMaker endpoint response times can differ from the response times observed in CloudWatch due to how and when the metrics are collected, aggregated, and reported. Although SageMaker queries CloudWatch for its metrics, the values displayed are not a direct reflection of individual request latencies.

Although absolute latency values differ across tools, this variation does not impact the primary objective of this research.

The forecasted response times provide insight into the Lambda function's performance, while CPU time reflects the processing demand, and memory utilization indicates the function's memory load. Over the next five forecasted steps, all three metrics show stability, suggesting that the API is handling incoming requests efficiently without any noticeable performance degradation. Below figure (SageMaker used Plotly [15] to plot the graph) show the actual values of all three metrics and predicted values.



Historicalvspredicted.png

Fig. 14. Historical vs Predicted Metrics

VII. CONCLUSION

AWS Lambda function which is known for its serverless architecture helps to autoscale dynamically based on workload without manual infrastructure management. Performance test data generated by JMeter was used rather than production data to have a safe and controlled environment. Additionally, AWS SageMaker offered an efficient platform for developing, training, and evaluating predictive models.

In this study, observed stability in response time, CPU usage, and memory utilization which indicates that the function is currently capable of handling the existing workload efficiently without a need of immediate intervention. Forecasting performance metrics using ML models provide valuable

insights into the near-future behavior of the applications (in this case, Lambda function) to anticipate performance issues before they occur, prevent server crashes, and make informed decisions about when to scale resources.

Combining predictive analytics with serverless computing and auto scaling, organizations can achieve improved reliability, cost efficiency, and proactive performance management of the applications.

VIII. FUTURE WORK

Future enhancements of this study will focus on improving the robustness, accuracy, and practical usability of the ML model. One key area of improvement is extending the model's capability to predict long-term future values, rather than being limited to near-term forecasts. Also, planning to conduct long hours tests to evaluate the model's performance and stability under extended operational conditions. Another significant extension is the integration of an automated alerting mechanism. By defining threshold values for response time, CPU usage, and memory utilization, the system can generate alerts whenever predicted metrics exceed acceptable limits. Additionally, the model can be enhanced to make predictions based on a broader historical context, rather than relying solely on the current sliding window.

Together, these improvements would transform the model into a more comprehensive and intelligent performance forecasting and alerting system, supporting long-term capacity planning, cost optimization, and reliable serverless application management.

IX. ACKNOWLEDGMENTS

The authors used ChatGPT to assist with pseudocode generation, grammar checking, and language refinement during the preparation of this manuscript. The experiments and analyses presented in this study were performed using custom Python scripts. The complete Python code supporting the simulation results, including data generation and visualization routines, is available from the corresponding author upon reasonable request.

REFERENCES

- [1] Amazon Web Services. *Amazon API Gateway*. <https://docs.aws.amazon.com/apigateway/>. AWS Documentation. 2025.
- [2] Amazon Web Services. *Amazon CloudWatch*. <https://docs.aws.amazon.com/cloudwatch/>. AWS Documentation. 2025.
- [3] Amazon Web Services. *Amazon DynamoDB*. <https://docs.aws.amazon.com/amazondynamodb/>. AWS Documentation. 2025.
- [4] Amazon Web Services. *Amazon SageMaker*. <https://docs.aws.amazon.com/sagemaker/>. AWS Documentation. 2025.
- [5] Amazon Web Services. *Amazon SageMaker Studio and Jupyter Notebooks*. <https://docs.aws.amazon.com/sagemaker/latest/dg/notebooks.html>. AWS Documentation. 2025.
- [6] Amazon Web Services. *AWS Lambda*. <https://docs.aws.amazon.com/lambda/>. AWS Documentation. 2025.
- [7] Apache Software Foundation. *User's Manual: Building a Web Test Plan*. Retrieved September 02, 2025. 2025. URL: <https://jmeter.apache.org/usermanual/build-web-test-plan.html>.
- [8] Ziji Chen et al. *Detecting Anomalies in Machine Learning Infrastructure via Hardware Telemetry*. 2025. arXiv: 2510.26008 [cs.PF]. URL: <https://arxiv.org/abs/2510.26008>.
- [9] Sowmya Chintakindi. "The Hidden Business Costs of Ignoring Performance Testing-The Silent Budget Killer". In: *PESARO 2025 : The Fifteenth International Conference on Performance, Safety and Robustness in Complex Systems and Application*. ThinkMind Digital Library, 2025, pp. 22–27. ISBN: 978-1-68558-280-7.
- [10] Sowmya Chintakindi and Shiva Shankar Kusuma. "Integrating Performance Testing Pipelines with Cloud-Based Analytics for Predictive Modeling Using ML". In: *2025 IEEE 16th Annual Ubiquitous Computing, Electronics Mobile Communication Conference (UEMCON)*. 2025, pp. 0757–0765. DOI: 10.1109/UEMCON67449.2025.11267575.
- [11] Python Software Foundation. *Python Programming Language*. <https://www.python.org/>. Accessed: 2025-10-10. 2023.
- [12] E. B. Halili. *Apache JMeter: A Practical Beginner's Guide to Automated Testing and Performance Measurement for Your Websites*. Packt Publishing, 2008.
- [13] Tim Januschowski et al. "A Flexible Forecasting Stack". In: *Proc. VLDB Endow.* 17.12 (Aug. 2024), 3883–3892. ISSN: 2150-8097. DOI: 10.14778/3685800.3685813. URL: <https://doi.org/10.14778/3685800.3685813>.
- [14] Adam Paszke et al. "PyTorch: an imperative style, high-performance deep learning library". In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [15] Plotly Technologies Inc. *Plotly Python Graphing Library*. <https://plotly.com/python/>. Online documentation. 2025.
- [16] Naga Surya Teja Thallam. "AI-Powered Monitoring and Predictive Maintenance for Cloud Infrastructure: Leveraging AWS Cloud Watch and ML". In: *International Journal of Artificial Intelligence, Data Science, and Machine Learning* 6.1 (2025), pp. 55–61.
- [17] J. Wen et al. "SuperFlow: Performance Testing for Serverless Computing". In: *arXiv preprint arXiv:2306.01620* (2023).